

Tema 3 — Etiquetado Morfosintáctico

(POS Tagging)

Procesamiento del Lenguaje Natural — UNIR | Guía de estudio profunda con derivaciones matemáticas

Material de apoyo para examen final | Abril 2026

 **Cómo usar esta guía:** Contiene todo el contenido del Tema 3 con derivaciones matemáticas completas, analogías, dudas frecuentes resueltas y resumen rápido al final. Está pensada para usarse como material de apoyo durante el examen.

1. Categorías Morfosintácticas (POS)

Definición — Morfosintaxis: Parte de la gramática que integra la *morfología* (estructura de las palabras) y la *sintaxis* (modo en que se combinan). Las categorías morfosintácticas clasifican las palabras según su tipo gramatical — llamadas **POS (Part-Of-Speech)** en inglés.

La clasificación clásica del español define **9 categorías gramaticales**:

Categoría	Descripción	Ejemplo
Sustantivo / Nombre	Posee género y número; designa entidades	perro, ciudad
Determinante	Determina al sustantivo; posición prenominal	el, este, un
Adjetivo	Modifica al sustantivo; denota cualidades	inteligente, rojo
Pronombre	Sustituye al sustantivo o sintagma nominal	ella, esto, quién
Verbo	Variación de persona, número, tiempo, modo	correr, es
Adverbio	Invariable; modifica verbo, adjetivo u oración	rápidamente, muy
Preposición	Invariable; introduce términos nominales	de, en, por
Conjunción	Invariable; coordina o subordina	y, pero, aunque
Interjección	Enunciados exclamativos, impresiones	¡ay!, ¡oye!

💡 **¿Por qué son útiles las categorías POS?** Conocer la categoría de una palabra informa sobre sus vecinas. Si una palabra es sustantivo, probablemente va precedida de determinante o seguida de adjetivo. Esto es exactamente lo que explotarán los HMM.

El Penn Treebank

Para el inglés, el estándar más usado es el **Penn Treebank** (Marcus et al., 1993): un conjunto de **45 etiquetas** que identifican las partes de la oración con mayor granularidad.

Etiqueta	Descripción	Ejemplo	Etiqueta	Descripción	Ejemplo
NN	Noun, singular	llama	VB	Verb base form	eat
NNS	Noun, plural	llamas	VBD	Verb past tense	ate
NNP	Proper noun, sing.	IBM	VBG	Verb gerund	eating
DT	Determiner	a, the	VBZ	Verb 3sg pres	eats
JJ	Adjective	yellow	MD	Modal	can, should
IN	Preposition	of, in, by	RB	Adverb	quickly
PRP	Personal pronoun	I, you, he	CC	Coord. conjunction	and, but

2. Etiquetado Morfosintáctico (POS Tagging)

Definición — POS Tagging: Proceso de asignar una etiqueta de categoría gramatical a cada palabra de un texto. Entrada: secuencia de palabras. Salida: secuencia de pares (palabra, etiqueta).

Ejemplo — Desambiguación:

"Bebo un vaso del **vino** tinto" → vino/NN (sustantivo)

"**Vino** de un lugar lejano" → vino/VBZ (verbo)

La misma palabra recibe etiquetas distintas según el **contexto**. Esto se llama **desambiguación léxica**.

Preprocesado: Tokenización

Antes del POS tagging se realiza la **tokenización**: separar la oración en tokens (unidades mínimas). Por ejemplo, la contracción "del" se separa en dos tokens: `de/IN` y `el/DT`.

Flujo completo del texto: Texto crudo → Tokenización → POS Tagging → Análisis sintáctico → Representación semántica

3. Modelos Ocultos de Markov (HMM) para POS Tagging

Definición — HMM (Hidden Markov Model): Modelo estadístico representable como una máquina de estados finitos donde las transiciones entre estados son *probabilísticas* y los estados son *no observables directamente*. El objetivo es inferir los estados ocultos a partir de las observaciones.

En POS Tagging:

- **Estados ocultos** = etiquetas gramaticales (NN, VB, DT...)
- **Observaciones** = palabras del texto

3.1 La Pregunta Fundamental

Dada una secuencia de n palabras $w_1^n = w_1, w_2, \dots, w_n$,

queremos encontrar la secuencia de etiquetas $\hat{t}_1^n$ más probable:

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} P(t_1^n \mid w_1^n)$$

Notación importante:

- El **sombrero** $\hat{}$ indica "valor estimado / mejor candidato"
- El subíndice 1 y superíndice n indican "desde posición 1 hasta posición n"
- **argmax** es un operador (no función estricta) que devuelve el *argumento* que maximiza la expresión — en este caso la secuencia de etiquetas, no el valor numérico de la probabilidad

3.2 Aplicación de la Regla de Bayes

Aplicando la regla de Bayes:

$$P(t_1^n \mid w_1^n) = \frac{P(w_1^n \mid t_1^n) \cdot P(t_1^n)}{P(w_1^n)}$$

Como buscamos el argmax respecto a t_1^n , el denominador $P(w_1^n)$

es una **constante positiva** — las palabras observadas no cambian entre las secuencias que comparamos. Por tanto:

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} P(w_1^n \mid t_1^n) \cdot P(t_1^n)$$

💡 **Duda frecuente de sesión:** ¿Por qué eliminar $P(w_1^n)$ no es "quitar una repetición"?

$P(w_1^n)$ no aparece en el numerador. Su eliminación se justifica

porque es una constante respecto a la optimización — no afecta *cuál* secuencia de etiquetas es la mejor, solo *qué tan alto* es el valor máximo. Analogía: si tres personas compiten en altura paradas sobre la misma silla, la silla no cambia quién es el más alto — podés ignorarla para decidir el ganador.

3.3 Las Dos Suposiciones Simplificadoras

Sin simplificaciones, calcular esto requeriría evaluar $45^{10} \approx 34$ billones

de combinaciones para una frase de 10 palabras con 45 etiquetas. Los HMM hacen dos suposiciones que lo vuelven tratable:

Suposición 1 — Independencia de emisión:

La probabilidad de una palabra depende *solo* de su propia etiqueta, independientemente de las vecinas:

$$P(w_1^n \mid t_1^n) \approx \prod_{i=1}^n P(w_i \mid t_i)$$

Suposición 2 — Propiedad de Markov (bigrama):

La probabilidad de una etiqueta depende *solo* de la etiqueta inmediatamente anterior:

$$P(t_1^n) \approx \prod_{i=1}^n P(t_i \mid t_{i-1})$$

Esta suposición convierte el costo de $O(45^n)$ en lineal

$O(n)$ respecto a la longitud de la frase.

3.4 La Ecuación Completa del Etiquetador HMM

Combinando ambas suposiciones:

$$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} \prod_{i=1}^n P(w_i | t_i) \cdot P(t_i | t_{i-1})$$

Factor	Nombre formal	Pregunta que responde	Captura...
$P(t_i t_{i-1})$	Probabilidad de transición (Matriz A)	¿Qué etiqueta sigue a otra etiqueta?	Estructura sintáctica del lenguaje
$P(w_i t_i)$	Probabilidad de emisión (Matriz B)	¿Qué palabra genera una etiqueta?	Asociación léxica palabra-categoría

 **Duda frecuente de sesión:** ¿Por qué se usa logaritmo en la práctica?

Multiplicar muchas probabilidades (valores entre 0 y 1) produce **underflow numérico** — números tan pequeños que la computadora los redondea a cero. La solución es aplicar logaritmo, convirtiendo el producto en suma:

$$\operatorname{argmax}_t \prod_i P(w_i | t_i) \cdot P(t_i | t_{i-1}) = \operatorname{argmax}_t \sum_i [\log P(w_i | t_i) + \log P(t_i | t_{i-1})]$$

El argmax no cambia porque el logaritmo es monotónamente creciente: si

$$a > b \quad , \text{ entonces } \quad \log(a) > \log(b) \quad .$$

4. Estimación de Parámetros — Máxima Verosimilitud (MLE)

Los parámetros del HMM se estiman a partir de un **corpus lingüístico etiquetado** — una colección de textos donde cada palabra ya tiene su etiqueta anotada por expertos. El corpus estándar usado en el tema es el **WSJ (Wall Street Journal)**, con 1 millón de palabras anotadas con etiquetas Penn Treebank.

MLE — Estimación de Máxima Verosimilitud: Método que estima los parámetros del modelo contando ocurrencias en el corpus de entrenamiento.

4.1 Probabilidad de Transición

$$P(t_i \mid t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

Donde $C(\cdot)$ denota el conteo en el corpus. **Ejemplo numérico real del corpus WSJ:**

$$P(VB \mid MD) = \frac{C(MD, VB)}{C(MD)} = \frac{10471}{13124} = 0.80$$

Interpretación: después de un verbo modal (como "can", "should"), hay un 80% de probabilidad de que siga un verbo en forma base. Esto tiene sentido lingüístico — "can run", "should eat".

4.2 Probabilidad de Emisión

$$P(w_i \mid t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

Ejemplo numérico real del corpus WSJ:

$$P(\text{will} \mid MD) = \frac{C(MD, \text{will})}{C(MD)} = \frac{4046}{13124} = 0.31$$

⚠ **Atención en el examen:** La probabilidad de emisión responde "si voy a generar una etiqueta MD, ¿qué tan probable es que emita la palabra 'will'?" — NO es la probabilidad a posteriori de que 'will' sea MD. Esa sería

$$P(MD \mid \text{will}) \quad , \text{ que es diferente.}$$

4.3 El Problema de Probabilidad Cero

Si una combinación (etiqueta, palabra) nunca apareció en el corpus, su conteo es 0 y por tanto su probabilidad de emisión es 0. Esto es catastrófico en Viterbi porque **un solo cero elimina toda la ruta** — el producto completo se vuelve 0.

Soluciones mediante técnicas de **suavizado (smoothing)**:

Técnica	Idea	Fórmula
Laplace (+1)	Sumar 1 a todos los conteos	$\frac{C(t_i, w_i) + 1}{C(t_i) + V}$
Good-Turing	Redistribuir masa de prob. de eventos frecuentes a raros	Compleja
Interpolación	Combinar unigramas, bigramas y trigramas	$\lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3$

Donde V es el tamaño del vocabulario. Este problema es especialmente grave con palabras **OOV (Out-Of-Vocabulary)** — palabras que no aparecen en el corpus de entrenamiento.

5. Algoritmo de Viterbi

Definición — Viterbi: Algoritmo de programación dinámica que encuentra eficientemente la secuencia de estados ocultos más probable en un HMM. Trabaja sobre una estructura de **trellis** (enrejado) — una matriz donde las columnas son palabras y las filas son etiquetas posibles.

5.1 Estructura del Trellis

Para una frase de T palabras y N etiquetas posibles, el trellis tiene

$N \times T$ celdas. Cada celda almacena:

- **Valor de Viterbi** $v_t(j)$: la probabilidad de la ruta más probable que llega al estado j en tiempo t
- **Backpointer:** qué estado en $t - 1$ produjo ese máximo (para reconstruir la ruta al final)

 **Analogía de sesión:** El backpointer es como el GPS que ya llegó al destino — guarda el rastro de qué camino tomó para llegar. Al finalizar, se hace un **backtrace** (recorrido inverso), similar conceptualmente al backpropagation pero sin ajuste de pesos — solo reconstrucción del camino óptimo.

5.2 Las Tres Fases del Algoritmo

Fase 1 — Inicialización (primera columna, primera palabra w_1):

$$v_1(j) = P(t_j \mid \text{start}) \cdot P(w_1 \mid t_j)$$

El estado inicial tiene probabilidad $v_0(\text{start}) = 1.0$.

Fase 2 — Recursión (columnas 2 a T):

$$v_t(j) = \max_{i=1}^N [v_{t-1}(i) \cdot P(t_j \mid t_i) \cdot P(w_t \mid t_j)]$$

El backpointer guarda el i que produjo ese máximo.

Fase 3 — Backtrace: Seguir los backpointers desde el estado final hasta el inicial para obtener la secuencia óptima de etiquetas.

Diferencia clave entre algoritmos:

Algoritmo	Operación en cada paso	Uso
Forward (filtrado)	Suma todas las rutas que llegan a un estado	Calcular probabilidad marginal
Viterbi	Conserva solo la ruta de máxima probabilidad	Encontrar la mejor secuencia

5.3 Ejemplo Completo — "Janet will back the bill"

La secuencia correcta de etiquetas es: Janet/NNP will/MD back/VB the/DT bill/NN

Columna 1 — "Janet"

"Janet" solo aparece como nombre propio en el corpus, por lo que:

$$v_1(NNP) = P(NNP | start) \cdot P(\text{Janet} | NNP) = 0.2767 \times 0.000032 = 0.000009$$

Todas las demás etiquetas tienen valor 0 para "Janet".

Columna 2 — "will" (palabra ambigua: MD, VB o NN)

$$v_2(MD) = v_1(NNP) \cdot P(MD | NNP) \cdot P(\text{will} | MD)$$

$$v_2(MD) = 0.000009 \times 0.0110 \times 0.308431 = 0.0000000305$$

Comparando los tres caminos posibles:

Camino	Transición $P(t NNP)$	Emisión $P(\text{will} t)$	Valor Viterbi
NNP → MD	0.0110	0.308431	3.05×10^{-8} ✓
NNP → VB	0.0009	0.000028	2.27×10^{-13}
NNP → NN	0.0584	0.000200	1.05×10^{-9}

💡 **Insight clave:** NN tiene mejor transición desde NNP ($0.0584 > 0.0110$), pero su emisión es brutalmente pequeña (0.000200 vs 0.308431). Viterbi evalúa el *producto* de ambos — ninguno factor domina solo. Esto demuestra la elegancia del modelo: la estructura sintáctica y la asociación léxica se balancean mutuamente.

Se continúa columna por columna hasta "bill", luego el backtrace reconstruye la secuencia óptima: **NNP → MD → VB → DT → NN**.

6. POS Tagging con Aprendizaje Automático

Los HMM tienen una limitación importante: la suposición bigrama solo mira la etiqueta *anterior*. Los modelos de aprendizaje automático superan esto con ventanas de contexto más amplias.

Enfoque discriminativo: En lugar de modelar probabilidades de generación, aprenden directamente una función de clasificación:

$$f(w_{i-2}, w_{i-1}, w_i, w_{i+1}, w_{i+2}, t_{i-2}, t_{i-1}, \dots) = t_i$$

Pueden considerar palabras *anteriores* Y *posteriores* simultáneamente.

Modelo	Referencia	Característica clave
Perceptrón	Collins, 2002	Aprende pesos lineales para features
Log-lineal bidireccional	Toutanova et al., 2003	Contexto izquierdo Y derecho simultáneo
SVM (Support Vector Machine)	Giménez y Márquez, 2004	Maximiza el margen entre clases
CRF (Conditional Random Field)	Lafferty, 2001	Modela secuencias completas con features ricas

Features Típicas en Modelos Supervisados

Tipo de feature	Ejemplo	Por qué es útil
Palabras vecinas	$w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}$	Contexto local
Etiquetas previas	t_{i-1}, t_{i-2}	Coherencia sintáctica

Prefijos/ sufijos	"-mente" → adverbio	Morfología predictiva
Mayúscula inicial	"Madrid" → NNP	Nombres propios
Bigramas y trigramas	pares y tríos de palabras	Patrones frecuentes
Longitud	número de caracteres	Discrimina tipos de palabras

Modelos no supervisados: Existen también algoritmos que etiquetan clústers de palabras en clases morfosintácticas sin necesidad de datos etiquetados — especialmente útiles cuando no hay corpus anotados disponibles (Christodoulopoulos et al., 2010).

7. NER — Named Entity Recognition

Definición — NER (Reconocimiento de Entidades Nombradas): Tarea que identifica y clasifica menciones de entidades específicas dentro de un texto — personas, organizaciones, lugares, fechas, cantidades monetarias, etc.

Ejemplo:

[**PER** Miguel López], de [**ORG** Microsoft], ha ido a una conferencia en [**GPE** Londres] el [**DATE** lunes].

Categorías Principales

Etiqueta	Tipo	Ejemplo
PER	Persona	Frodo Baggins, María García
ORG	Organización / Empresa	Microsoft, ONU
GPE	País, ciudad, estado	Madrid, Colombia
LOC	Ubicación concreta	Vesubio, río Amazonas
MONEY	Referencia monetaria	\$5, €200
DATE	Fecha o período temporal	05/02/2021, jueves

Esquemas de Representación BIO / BIOES

Problema especial del NER: una entidad puede abarcar **múltiples tokens** ("United Airlines Holding" = 3 tokens, 1 entidad ORG). Se necesitan esquemas de etiquetado especiales:

Esquema	Prefijos	"Jane Villanueva"	"Chicago"
IO	I (inside), O (outside)	I-PER, I-PER	I-LOC
BIO	B (begin), I (inside), O	B-PER, I-PER	B-LOC

BIOES	B, I, O, E (end), S (single)	B-PER, E-PER	S-LOC
--------------	------------------------------	--------------	-------

💡 **¿Por qué BIO es mejor que IO?** IO no puede distinguir si dos entidades del mismo tipo están juntas. BIO lo resuelve: un nuevo B siempre indica el inicio de una nueva entidad, incluso si es del mismo tipo que la anterior.

Métodos para Resolver NER

Método	Descripción	Ventaja / Limitación
Diccionarios	Lista de entidades conocidas y su tipo	Simple pero no generaliza
Reglas	Patrones lingüísticos predefinidos	Interpretable pero frágil
ML / Estadístico	Aprende de corpus anotados con features de contexto	Robusto y generalizable

8. Resumen Rápido para el Examen

Los 5 conceptos más importantes del Tema 3:

1. **POS Tagging** = asignar etiqueta gramatical a cada token. Requiere desambiguación léxica.
2. **HMM** = estados ocultos (etiquetas) + observaciones (palabras) + probabilidades de transición A + emisión B.
3. **Viterbi** = programación dinámica sobre trellis. Tres fases: inicialización → recursión → backtrace. Guarda backpointers para reconstruir la ruta óptima.
4. **MLE** = los parámetros se estiman por conteo en corpus etiquetados. Problema: ceros → suavizado.
5. **NER** = extensión del POS tagging para identificar entidades nombradas. Esquema BIO es el estándar.

Fórmulas Clave

Nombre	Fórmula
Objetivo HMM	$\hat{t}_1^n = \operatorname{argmax}_{t_1^n} \prod_{i=1}^n P(w_i t_i) \cdot P(t_i t_{i-1})$
Transición MLE	$P(t_i t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$
Emisión MLE	$P(w_i t_i) = \frac{C(t_i, w_i)}{C(t_i)}$

Viterbi recursión	$v_t(j) = \max_i [v_{t-1}(i) \cdot P(t_j t_i) \cdot P(w_t t_j)]$
Laplace smoothing	$P(w_i t_i) = \frac{C(t_i, w_i) + 1}{C(t_i) + V}$

Tabla Comparativa Final

Aspecto	HMM	ML Supervisado (SVM, CRF...)
Contexto considerado	Solo etiqueta anterior (bigrama)	Ventana amplia (anterior y posterior)
Tipo de modelo	Generativo	Discriminativo
Features	Limitadas (transición + emisión)	Ricas (prefijos, sufijos, mayúsculas...)
Entrenamiento	Corpus etiquetado (conteo)	Corpus etiquetado (optimización)
Algoritmo de decodificación	Viterbi	Varía según modelo
Interpretabilidad	Alta (probabilidades directas)	Variable

Claves del Test de Autoevaluación:

- Q1: A, C (morfosintaxis = morfología + sintaxis; determina partes de la oración)
- Q2: A, B, C, D (todos son nombres equivalentes para lo mismo)
- Q3: B, D (transición = $P(\text{etiqueta actual} \mid \text{etiqueta anterior})$; MLE = $C(t-1, t)/C(t-1)$)
- Q4: A, B, C (Penn Treebank etiqueta signos de puntuación también)
- Q5: A, B, C (la entrada puede incluir preprocesado, no "solo palabras")
- Q6: B, C, D (HMM es modelo estadístico, autómata finito con transiciones probabilísticas)
- Q7: A, B, D (Viterbi estima probabilidades óptimas, no "fija condiciones")
- Q8: B, D (aplica desambiguación y tokenización; SÍ se ve afectado por ambigüedad)
- Q9: A, C (emisión: dado etiqueta $\rightarrow$ probabilidad de palabra; NO al revés)
- Q10: A, B, D (la prob. de Viterbi = $v(t-1) \times \text{transición} \times \text{observación del símbolo/palabra}$, no etiqueta)